

Transformers I: Secuencias y Atención

De secuencias a atención

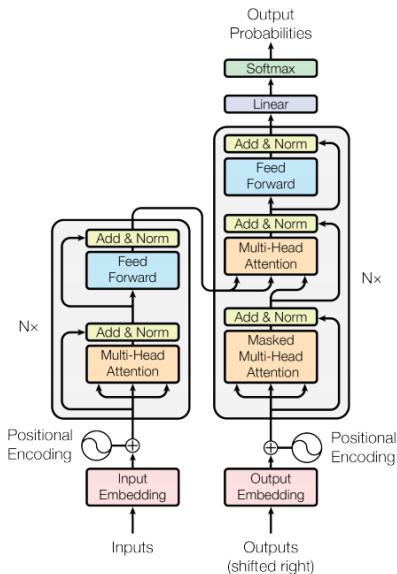
Cesar Garcia

2025

- Hasta ahora trabajamos con **datos espaciales** (CNN)
- Muchos problemas reales son **secuenciales**
- El orden importa: lenguaje, tiempo, series temporales
- **Hoy:** mecanismo de atención + preparación para Transformers

¿Qué significa que un modelo “entienda” una secuencia?

Attention is all you need



Qué es una secuencia

Definición

Una secuencia es:

- una lista ordenada de elementos
- con dependencias entre posiciones

Ejemplos:

- texto (palabras)
- audio (frames)
- precios (tiempo)

¿Por qué un MLP falla aquí?

De símbolos a vectores

Los modelos no operan sobre palabras directamente:

- tokenizamos
- cada token se convierte en un vector (embedding)

$$\text{token} \rightarrow \mathbb{R}^d$$

¿Qué captura un embedding bien entrenado?

Motivación

En lugar de procesar secuencialmente:

- cada token decide a qué otros tokens prestar atención

Atención = **routing de información**

¿Qué significa “mirar” a otro token?

Componentes

Para cada token calculamos:

- Query (Q): qué busco
- Key (K): qué ofrezco
- Value (V): qué transmito

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

¿Por qué necesitamos tres proyecciones distintas?

Scaled Dot-Product Attention

Fórmula

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

- 1 Similitud Q-K (producto punto)
- 2 Escalado por $\sqrt{d_k}$ (estabilidad)
- 3 Softmax (normalización)
- 4 Combinación ponderada de valores

¿Por qué escalamos por $\sqrt{d_k}$?

Self-Attention

En transformers:

- Q, K, V vienen de la **misma secuencia**
- cada token se contextualiza con los demás
- resultado: representación contextual de cada token

¿Qué gana un token al auto-atenderse?

Visualización de atención

Matriz de pesos

Cada fila muestra:

- a qué posiciones presta atención ese token
- valores más altos = mayor relevancia

	k0	k1	k2	k3	k4	k5	
q0	[.1	.05	.02	.8	.02	.01]	← token 0 mira principalmente
q1	[.2	.3	.3	.1	.05	.05]	← token 1 distribuido
q2	[...]						

Multi-Head Attention

De una cabeza a muchas

Una sola atención = un solo patrón.

Multi-head:

- divide d_{model} en h cabezas
- cada cabeza aprende un patrón diferente
- ejemplo: cabeza 1 $\rightarrow$ sintaxis, cabeza 2 $\rightarrow$ semántica

$$d_k = \frac{d_{model}}{h}$$

¿Qué ventaja tiene dividir la atención?

Proceso

- 1 Dividir Q , K , V en h cabezas
- 2 Atención independiente por cabeza
- 3 Concatenar resultados
- 4 Proyección de salida

$$\text{MHA}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

(Implementación completa en Sesión 14)

Problema: el orden

Limitación

La atención es **invariante al orden** por defecto.
Si permutamos tokens, el resultado es el mismo.

Solución:

- **Positional Encoding**

¿Qué se perdería si permutamos “el gato come” → “come gato el”?

Positional Encoding

Funciones sinusoidales

Agregamos información de posición al embedding:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

Luego: $X_{final} = E + PE$

¿Por qué seno y coseno?

Propiedades

- Frecuencias diferentes para cada dimensión
- Permite distinguir **posiciones relativas**
- Determinístico (no se aprende)
- Extrapola a secuencias más largas

Intuición: patrón único para cada posición, como una “huella digital”

Problema en generación

Al generar texto:

- token i **no debe ver** tokens futuros ($j > i$)
- de lo contrario, “haría trampa”

Solución: máscara triangular superior

$$\text{scores}[i, j] = \begin{cases} \text{score} & \text{si } j \leq i \\ -\infty & \text{si } j > i \end{cases}$$

Visualización

Sin máscara (encoder):

	k0	k1	k2	k3
q0	OK	OK	OK	OK
q1	OK	OK	OK	OK
q2	OK	OK	OK	OK
q3	OK	OK	OK	OK

Con máscara (decoder):

	k0	k1	k2	k3
q0	OK	X	X	X
q1	OK	OK	X	X
q2	OK	OK	OK	X
q3	OK	OK	OK	OK

Atención como mecanismo universal

Transformers funcionan porque:

- desacoplan posición y dependencia
- permiten paralelismo (vs RNN)
- modelan relaciones globales

No recorremos secuencias: las conectamos.

De bloques a arquitectura

Veremos:

- cómo ensamblar un transformer completo
- feedforward networks + residuales
- encoder y decoder completos
- **entrenar un modelo real**

Prepárate para construir tu propio transformer